

COMP0008: Computer Architecture & Concurrency

Lecture 8: Demo of the GCC MIPS Toolchain

Kevin Bryson

K.Bryson@ucl.ac.uk

Darwin Building Room 632.

Recommended Reading:

Patterson & Hennessey Section 2.12.

More advanced material in Appendix B.

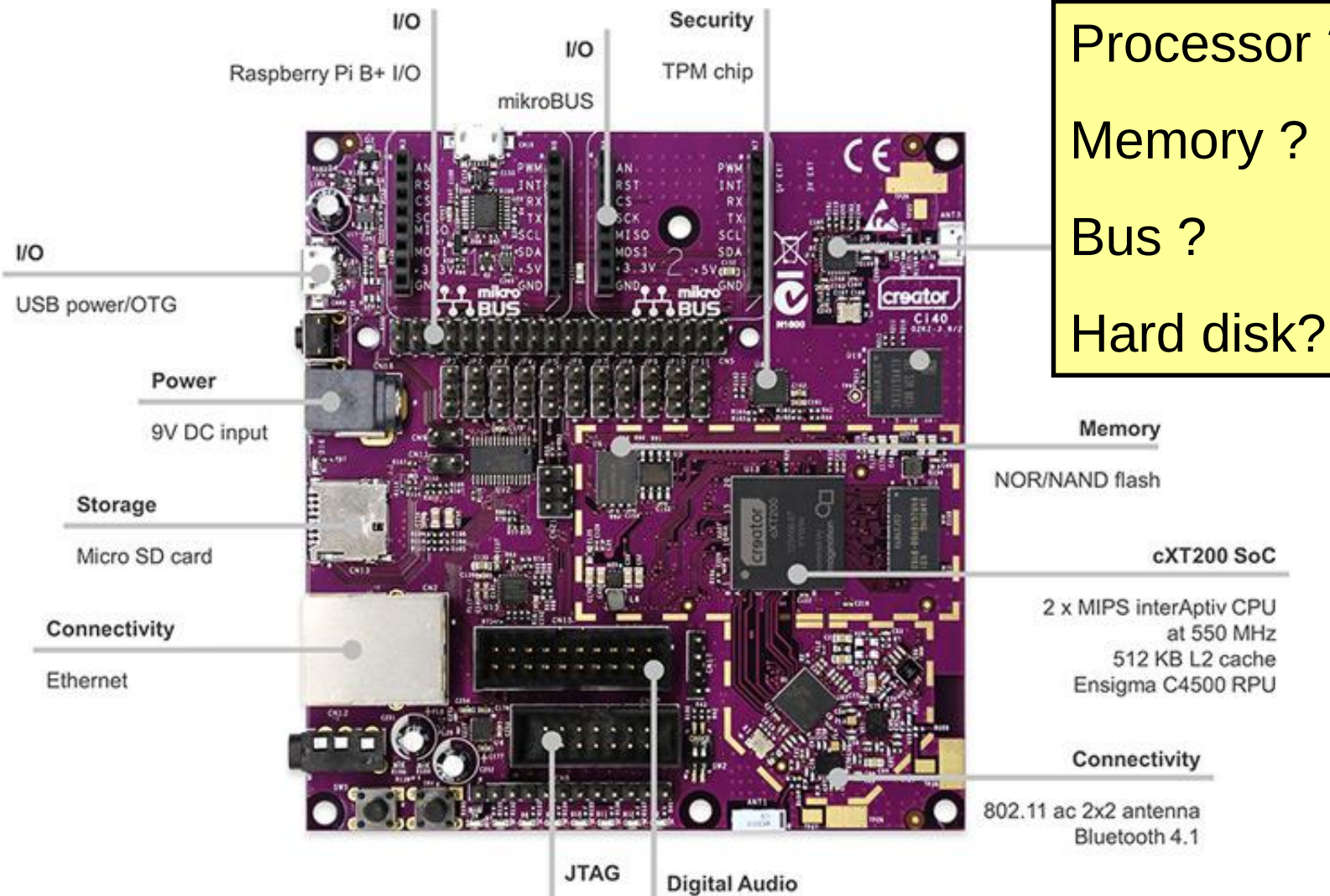
Acknowledgements:

Harris & Harris book for C-code and figs.

Overview

- We should now know about:
 - The key components of a computer.
 - The MIPS32 Instruction Set Architecture (ISA).
 - MIPS32 Assembly Language and encoding of this as machine code.
 - High-level language and its conversion to assembly.
- In this lecture I want to examine how real C language is compiled into real MIPS assembly language on Imaginations Ci40 single board computer.

The Imaginations Creative Ci40 single board computer - like a Raspberry Pi but with a MIPS32 processor



Harris & Harris has this example of high level C-code being compiled ...

the program into memory. The compilation process walks through these steps for a simple program.

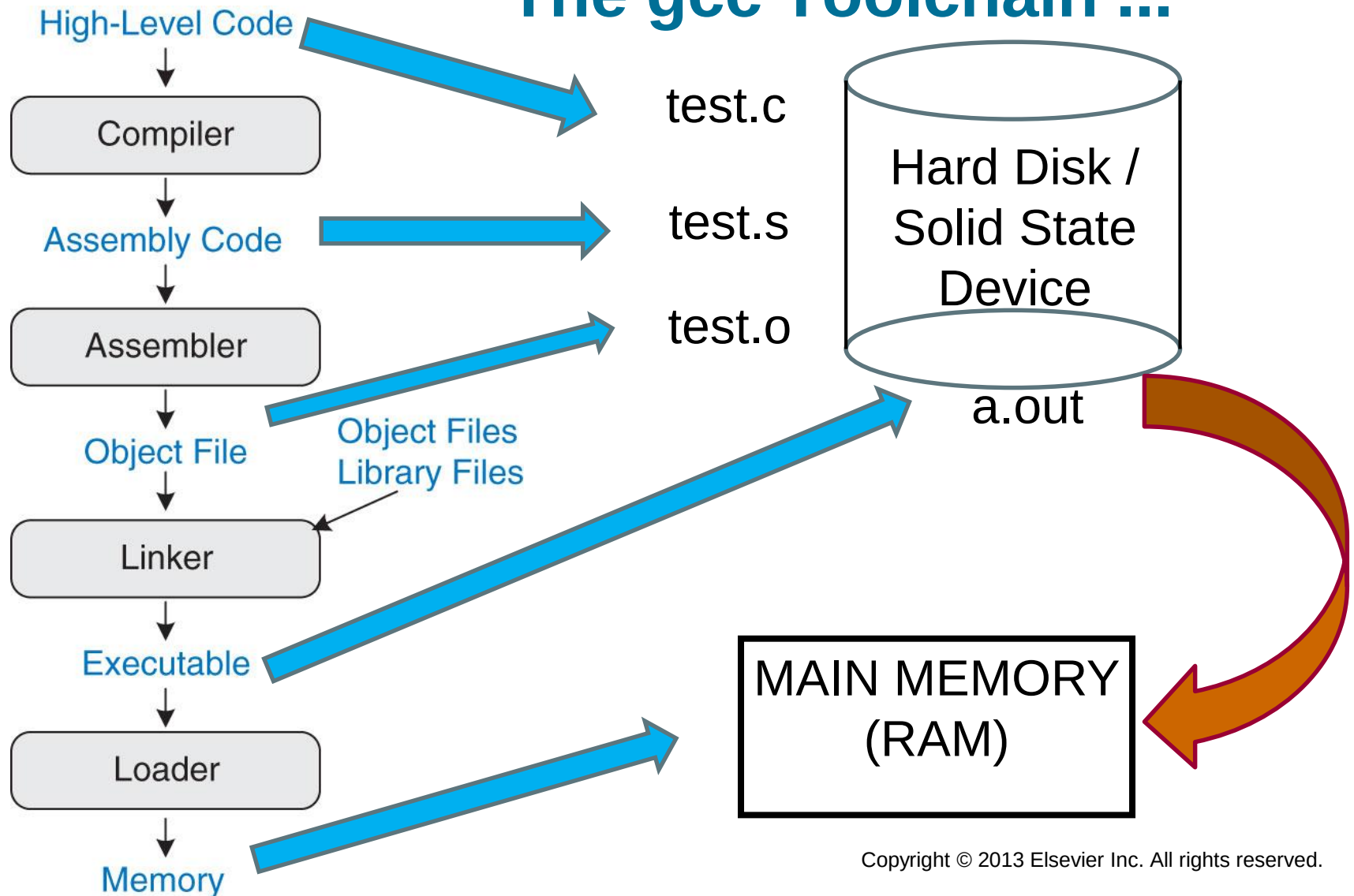
Step 1: Compilation
 A compiler translates high-level code into assembly language. Code Example 6.30 shows a simple high-level program with three global variables.

Code Example 6.30 COMPILING A HIGH-LEVEL PROGRAM

High-Level Code	MIPS Assembly Code
<pre> int f, g, y; // global variables int main (void) { f = 2; g = 3; y = sum (f, g); return y; } int sum (int a, int b) { return (a + b); } </pre>	<pre> .data f: g: y: .text main: addi \$sp, \$sp, -4 # make stack frame sw \$ra, 0(\$sp) # store \$ra on stack addi \$a0, \$0, 2 # \$a0 = 2 sw \$a0, f # f = 2 addi \$a1, \$0, 3 # \$a1 = 3 sw \$a1, g # g = 3 jal sum # call sum procedure sw \$v0, y # y = sum (f, g) lw \$ra, 0(\$sp) # restore \$ra from stack addi \$sp, \$sp, 4 # restore stack pointer jr \$ra # return to operating system sum: add \$v0, \$a0, \$a1 # \$v0 = a + b jr \$ra # return to caller </pre>

How is the executable produced?

The gcc Toolchain ...



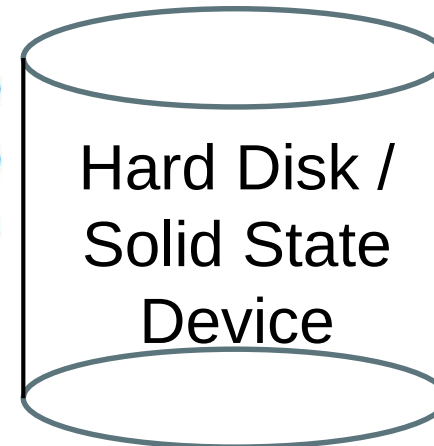
Binary Executable File on Disk

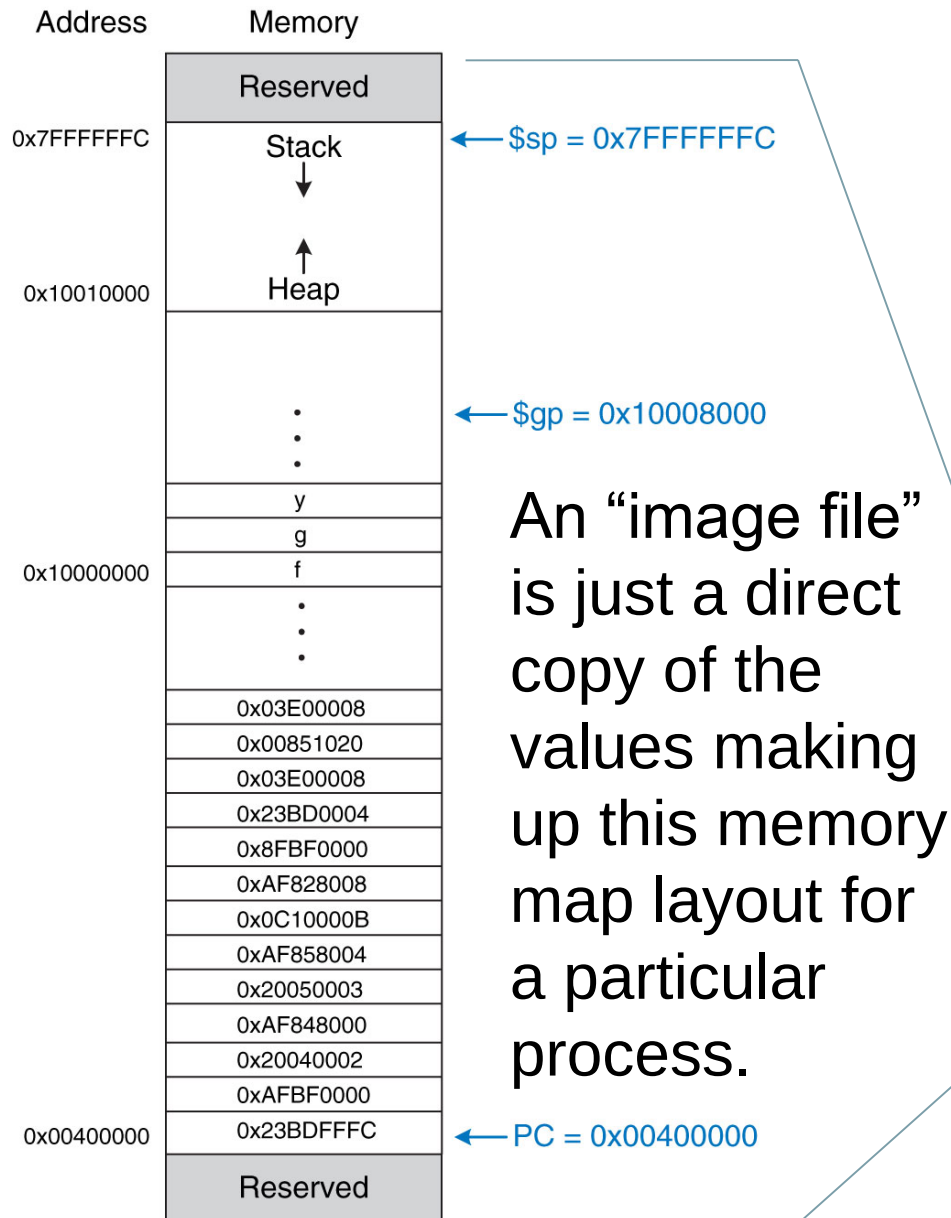
Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	Address	Instruction
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	Address	Data
	0x10000000	f
	0x10000004	g
	0x10000008	y

```

addi $sp, $sp, -4
sw  $ra, 0($sp)
addi $a0, $0, 2
sw  $a0, 0x8000($gp)
addi $a1, $0, 3
sw  $a1, 0x8004($gp)
jal  0x0040002C
sw  $v0, 0x8008($gp)
lw  $ra, 0($sp)
addi $sp, $sp, -4
jr  $ra
add $v0, $a0, $a1
jr  $ra

```

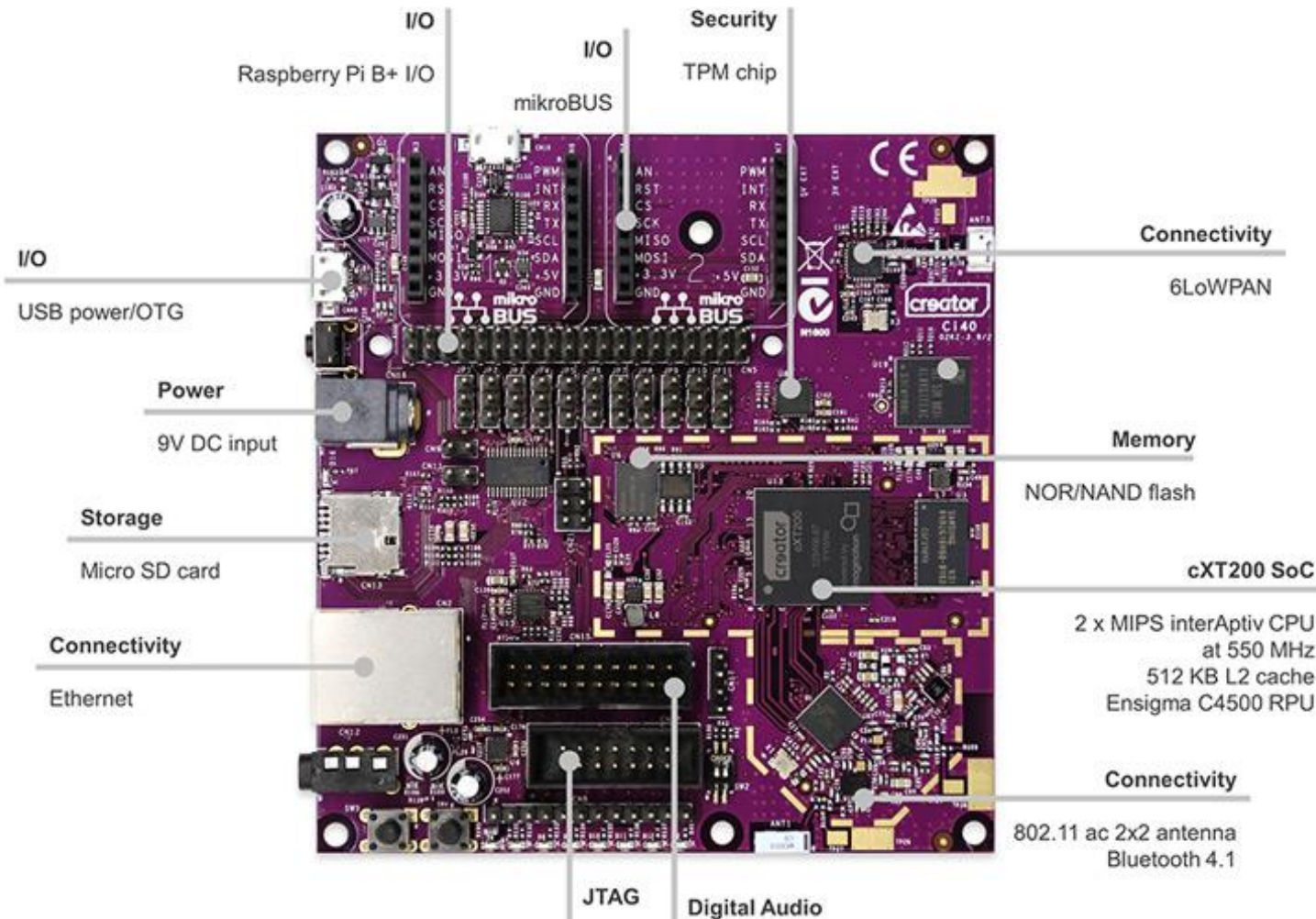




The OS calls a “loader” program to load the executable file into RAM following the MIPS memory map ... then it is executed

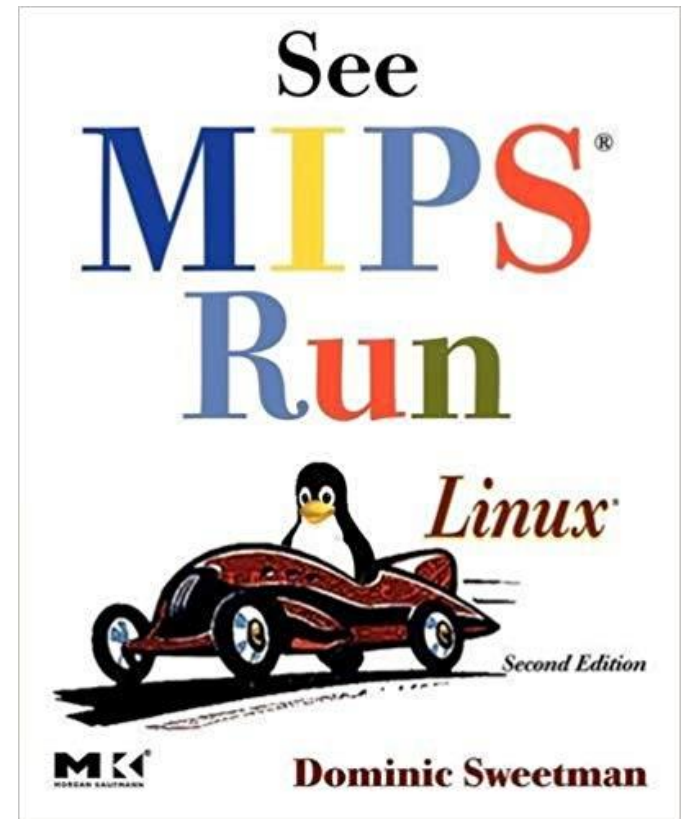
MAIN MEMORY (RAM)

Live Linux Demo of using “gcc -S” and “objdump -S” to look at real MIPS assembly generated by the gcc toolchain ...



(Optional) To fully understand how MIPS32 machine is working under Linux ...

- A full description of how the MIPS32 processor is working under Linux is given in this book.
- It also describes the full MIPS32 assembly language including the unusual assembly directives we have seen in the demo.
- Available from UCL library (both hardcopy and online)



Summary

- In this lecture I wanted to show you that real C language is compiled using a ‘toolchain’ in a ‘similar manner’ to what we have discussed previously ...
- But there is a lot more complexity in real machines in terms of the structure of object files, linking these together and loading the executable into the MIPS memory map.
- The assembly language generated (and assembly directives employed) are also much more complex and diverse in real systems.